

The $3 \times n$ holed-strip knight graph with two far
corners removed: open and closed tour existence
characterisation
(Lean 4 machine-checked claim of record)

Aurora0134*

September 30, 2026

Abstract

Let K_n be the knight graph of the $3 \times n$ chessboard with the two far corners $(0, 0)$ and $(2, n - 1)$ removed. We prove, with machine-checked Lean 4 proofs, the exact existence characterisation of tours on K_n : an open tour (Hamiltonian path) exists if and only if $n \in \{1, 4, 5\} \cup \{n \geq 8\}$, and a closed tour (Hamiltonian cycle) exists if and only if n is even and $n \geq 8$. Both theorems compile with no `sorry`, and `#print axioms` reports only `{propext, Quot.sound, Classical.choice}`. Four rounds of novelty review found no occupying record for the family: the nearest published neighbours remove one square from $3 \times n$ boards or two squares from $4 \times n$ boards, and none of them treats two-square $3 \times n$ boards. This note is a priority claim of record, not a full paper.

1 Introduction

We study the knight graph K_n of the $3 \times n$ chessboard with the two far corners removed — a deficient-board variant of the classical $3 \times n$ knight’s-tour problem whose undirected tour counts are registered as OEIS A169696 [9]. For this variant we establish the complete existence characterisation for both open and closed tours, formalised and proved in Lean 4 with every assertion adjudicated by the kernel. The result was produced by the AI4Math pipeline (LLM generation under Lean-kernel adjudication); this note is a

*Contact: via the GitHub account Aurora0134; ORCID placeholder — to be filled in by the author before any upload. No personal information is carried in this note.

priority claim of record, and the full narrative paper is prepared separately. All formal statements and proofs were checked by the Lean 4 kernel: every proof compiles with no `sorry`, and `#print axioms` reports only `{propxt, Quot.sound, Classical.choice}`.

2 Statement

Throughout, a board cell is a pair (r, c) with $r \in \{0, 1, 2\}$ and $c \in \{0, \dots, n-1\}$; K_n has as vertices the $3n-2$ cells other than $(0, 0)$ and $(2, n-1)$, two vertices being adjacent when they are a knight's move apart (row difference 1 and column difference 2, or row difference 2 and column difference 1). An open tour is a Hamiltonian path of K_n ; a closed tour is a Hamiltonian cycle. Both notions use the standard mathlib predicates [2] (`Walk.IsHamiltonian` and `Walk.IsHamiltonianCycle`).

Theorem 1 (open tours). *K_n has an open tour if and only if $n = 1$, or $n = 4$, or $n = 5$, or $n \geq 8$.*

Theorem 2 (closed tours). *K_n has a closed tour if and only if n is even and $n \geq 8$.*

The case $n = 1$ in Theorem 1 is the one-point graph: the empty walk is a (trivial) Hamiltonian path. This reading was fixed before the statement freeze (the pre-freeze correction F0b recorded in the task card, backed by a kernel probe); the closed-tour side is unaffected because `IsHamiltonianCycle` excludes the empty walk.

Listing 1: Frozen statement (verbatim lines 43–76 of the frozen statement file, sha256 96875db4...): board encoding, the knight graph, and the head of Theorem 1. The proof body placeholder lies outside the excerpt.

```

1  /-- 3×n 棋盘的格子坐标。 -/
2  abbrev Cell (n : ℕ) := Fin 3 × Fin n
3
4  /-- 棋盘格子（非洞）：删去 (0,0) 与 (2, -n1), 用 `.val` 编码（见文
   件头注释）。 -/
5  abbrev IsCell (n : ℕ) (p : Cell n) : Prop :=
6    ¬ (p.1.val = 0 ∧ p.2.val = 0) ∧ ¬ (p.1.val = 2 ∧ p.2.val + 1 =
   n)
7
8  /-- 顶点类型：-3n2 个非洞格子。 -/
9  def V (n : ℕ) := {p : Cell n // IsCell n p}
10

```

```

11 instance (n : ℕ) : Fintype (V n) := Subtype.fintype _
12 instance (n : ℕ) : DecidableEq (V n) := Subtype.instDecidableEq
13
14 /-- 骑士走法的四个代表方向  $(b - a \in \{(1,2), (2,1), -(1,2), -(2,1)\})$ 
15   `fromRel` 自动对称化, 补全另外四个方向  $((/\Delta行/, /\Delta列/) \in \{(1,2),$ 
16    $(2,1)\}$  全 8 组合)。 -/
17 abbrev leap {n : ℕ} (a b : Cell n) : Prop :=
18   ((a.1 : ℕ) + 1 = (b.1 : ℕ) ∧ (a.2 : ℕ) + 2 = (b.2 : ℕ)) ∨
19   ((a.1 : ℕ) + 2 = (b.1 : ℕ) ∧ (a.2 : ℕ) + 1 = (b.2 : ℕ)) ∨
20   ((b.1 : ℕ) + 1 = (a.1 : ℕ) ∧ (a.2 : ℕ) + 2 = (b.2 : ℕ)) ∨
21   ((b.1 : ℕ) + 2 = (a.1 : ℕ) ∧ (a.2 : ℕ) + 1 = (b.2 : ℕ))
22 /-- 完整 8 向 (对称) 骑士关系。 -/
23 abbrev leap8 {n : ℕ} (a b : Cell n) : Prop := leap a b ∨ leap b a
24
25 /--  $K_n$ :  $3 \times n$  棋盘删  $(0,0)$ 、 $(2,-n1)$  后的骑士图。 -/
26 def knightGraph (n : ℕ) : SimpleGraph (V n) :=
27   SimpleGraph.fromRel (fun a b : V n => leap8 a.1 b.1)
28
29 /-- **定理 O** (开巡游, 命题 (a) 前半):  $K_n$  存在哈密顿路径
30   当且仅当  $n \in \{4, 5\} \cup \{n \geq 8\}$ 。
31    $n=1$  为单点图平凡路径情形 (见文件头修正注记)。 -/
32 theorem exists_hamiltonian_path_iff (n : ℕ) :
33   ( $\exists a b : V n, \exists w : (\text{knightGraph } n).\text{Walk } a b, w.\text{IsHamiltonian}$ )
34    $\leftrightarrow$ 
35    $n = 1 \vee n = 4 \vee n = 5 \vee 8 \leq n := \text{by}$ 

```

Listing 2: Frozen statement, verbatim lines 79–83: the head of Theorem 2.

```

1 /-- **定理 C** (闭巡游, 命题 (a) 后半):  $K_n$  存在哈密顿圈
2   当且仅当  $n$  为偶数且  $n \geq 8$ 。 -/
3 theorem exists_hamiltonian_cycle_iff (n : ℕ) :
4   ( $\exists a : V n, \exists w : (\text{knightGraph } n).\text{Walk } a a, w.$ 
5      $\text{IsHamiltonianCycle}$ )  $\leftrightarrow$ 
6      $\text{Even } n \wedge 8 \leq n := \text{by}$ 

```

3 Proof

The complete machine proof is the file `stage4b-full.lean` (2,633 lines), shipped byte-exactly in `proofs/`; the listings below are verbatim contiguous excerpts, in the order in which they appear in that file. In outline: the positive side is an induction machine on explicit vertex lists — closed

tours for even $n \geq 8$ by four explicit base lists ($n = 8, 10, 12, 14$) plus an insertion step that splices a fixed 8-column wedge into a designated break edge (strong induction over $n \bmod 8$); open tours for odd $n \geq 9$ by two base lists ($n = 9, 11$) plus a 4-column step; the small cases $n = 1, 4, 5$ by explicit witness lists whose adjacency, coverage and non-repetition are checked by `decide`. The negative side is degree and parity arguments for $n \leq 4$ and the odd closed case, plus — for the two hard zero values $n = 6, 7$ — a cut barrier: a general lemma stating that if deleting a vertex list S leaves more than $|S| + 1$ labelled blocks, then no Hamiltonian path exists, applied to explicit certificates whose correctness is decided by the kernel. The two final theorems assemble these pieces by case analysis on n (`omega` splits $n < 8$ into $0, \dots, 7$).

Listing 3: Verbatim lines 1589–1644: the closed-tour induction assembly — `closed_good_list` (strong induction, bases 8/10/12/14, step $n \mapsto n + 8$) and `closed_cycle_exists` (loading the good list as a Hamiltonian cycle).

```

1  /-- 闭侧好列表存在性: 偶数  $n \geq 8 \Rightarrow$  存在 GoodCycleList
2    (强归纳: 小  $n$  四基座 8/10/12/14, 步进  $m \mapsto m+8$ )。 -/
3  theorem closed_good_list :  $\forall n : \mathbb{N}, 8 \leq n \rightarrow \text{Even } n \rightarrow \exists l : \text{List}$ 
4    (V n), GoodCycleList l := by
5    intro n
6    apply Nat.strong_induction_on
7    (p := fun n =>  $8 \leq n \rightarrow \text{Even } n \rightarrow \exists l : \text{List (V n),}$ 
8      GoodCycleList l)
9    intro n ih hn heven
10   rcases Nat.lt_or_ge n 16 with h16 | h16
11   · -- 小  $n$ :  $8 \leq n < 16$  且偶数  $\Rightarrow n \in \{8, 10, 12, 14\}$ 
12     rcases heven with ⟨m, hm⟩
13     have hn' :  $n = 8 \vee n = 10 \vee n = 12 \vee n = 14$  := by omega
14     rcases hn' with h | h | h | h
15     · subst h; exact ⟨baseV8, baseV8_good⟩
16     · subst h; exact ⟨baseV10, baseV10_good⟩
17     · subst h; exact ⟨baseV12, baseV12_good⟩
18     · subst h; exact ⟨baseV14, baseV14_good⟩
19   · --  $n \geq 16$ : 从  $n - 8$  步进
20     have hn8 :  $n - 8 < n$  := by omega
21     have hn8' :  $8 \leq n - 8$  := by omega
22     have heven8 : Even (n - 8) := by
23       rcases heven with ⟨m, hm⟩
24       exact ⟨m - 4, by omega⟩
25     obtain ⟨l, hg⟩ := ih (n - 8) hn8 hn8' heven8
26     obtain ⟨l', hg'⟩ := step_closed_list hn8' hg
27     rw [← show n - 8 + 8 = n from by omega]

```

```

26     exact ⟨l', hg'⟩
27
28 /-- 闭巡游存在性（正向）：偶数  $n \geq 8 \Rightarrow K_n$  存在哈密顿圈
29   （GoodCycleList 经 cycleOfChain 装载为 IsHamiltonianCycle）。
30   -/
31 theorem closed_cycle_exists :  $\forall n : \mathbb{N}, \text{Even } n \rightarrow 8 \leq n \rightarrow$ 
32    $\exists a : V \ n, \exists w : (\text{knightGraph } n).\text{Walk } a \ a, w.$ 
33   IsHamiltonianCycle := by
34   intro n heven hn
35   obtain ⟨l, hg⟩ := closed_good_list n hn heven
36   obtain ⟨hne, hnd, hcov, hchain, hclose, j, hj1r, hj1c, hj2r,
37     hj2c⟩ := hg
38   obtain ⟨v1, v2, v3, h12, h13, h23⟩ := three_vertices_exist hn
39   have hlen :  $3 \leq l.\text{length}$  := by
40     have hsub :  $[v1, v2, v3] \subseteq l$  := fun a ha => by
41       rw [List.mem_cons, List.mem_cons, List.mem_singleton] at ha
42       rcases ha with h | h | h
43       · rw [h]; exact hcov v1
44       · rw [h]; exact hcov v2
45       · rw [h]; exact hcov v3
46   have hnd3 :  $([v1, v2, v3] : \text{List } (V \ n)).\text{Nodup}$  := by
47     rw [List.nodup_cons, List.nodup_cons]
48     refine ⟨?, ?, List.nodup_singleton v3⟩
49     · intro h
50       rw [List.mem_cons, List.mem_singleton] at h
51       rcases h with h | h
52       · exact h12 h
53       · exact h13 h
54     · intro h
55       rw [List.mem_singleton] at h
56       exact h23 h
57   have := (hnd3.subperm hsub).length_le
58   simpa using this
59   exact cycleOfChain l hne hnd hlen hcov hchain (hclose hne)

```

Listing 4: Verbatim lines 2001–2036: the open-tour induction assembly for odd $n \geq 9$ (bases 9/11, step $n \mapsto n + 4$).

```

1 /-! ### 12.5 奇归纳组装（T3a 收官） -/
2
3 /-- 开侧好列表存在性：奇数  $n \geq 9 \Rightarrow$  存在 GoodOpenList
4   （强归纳：小  $n$  两基座 9/11，步进  $m \mapsto m+4$ ）。 -/
5 theorem open_good_list :  $\forall n : \mathbb{N}, 9 \leq n \rightarrow \text{Odd } n \rightarrow \exists l : \text{List } (V$ 
6    $n), \text{GoodOpenList } l$  := by
7   intro n

```

```

7   apply Nat.strong_induction_on
8   (p := fun n => 9 ≤ n → Odd n → ∃ l : List (V n),
9     GoodOpenList l)
10  intro n ih hn hodd
11  rcases Nat.lt_or_ge n 13 with h13 | h13
12  · -- 小 n: 奇数且 9 ≤ n < 13 ⇒ n ∈ {9, 11}
13    have hn' : n = 9 ∨ n = 11 := by
14      rcases hodd with ⟨m, hm⟩
15      omega
16    rcases hn' with h | h
17    · subst h; exact ⟨openBase9V, openBase9V_good⟩
18    · subst h; exact ⟨openBase11V, openBase11V_good⟩
19  · -- n ≥ 13: 从 n - 4 步进
20    have hn4 : n - 4 < n := by omega
21    have hn4' : 9 ≤ n - 4 := by omega
22    have hodd4 : Odd (n - 4) := by
23      rcases hodd with ⟨m, hm⟩
24      exact ⟨m - 2, by omega⟩
25    obtain ⟨l, hg⟩ := ih (n - 4) hn4 hn4' hodd4
26    obtain ⟨l', hg'⟩ := step_open_list hn4' hg
27    rw [← show n - 4 + 4 = n from by omega]
28    exact ⟨l', hg'⟩
29  /-- 开巡游存在性 (正向·奇): 奇数 n ≥ 9 ⇒ K_n 存在哈密顿路径
30    (GoodOpenList 经 pathOfChain 装载为 IsHamiltonian)。 -/
31  theorem open_tour_exists_odd : ∀ n : ℕ, 9 ≤ n → Odd n →
32    ∃ a b : V n, ∃ w : (knightGraph n).Walk a b, w.IsHamiltonian
33    := by
34    intro n hn hodd
35    obtain ⟨l, hg⟩ := open_good_list n hn hodd
36    obtain ⟨hne, hnd, hcov, hchain, hend⟩ := hg
37    exact pathOfChain l hne hnd hcov hchain

```

Listing 5: Verbatim lines 2447–2575: the cut barrier. The generic lemmas `cut_card_bound` (label count along a chain), `cut_bound` and `no_hamiltonian_of_cut`, then the explicit $n = 6$ certificate (6 middle-column vertices, 8 blocks) and $n = 7$ certificate (5 vertices, 7 blocks). The two `set_option` resource-relaxation lines preceding this section are omitted from the excerpt; the full file carries them.

```

1  /-! ## 15. 割集屏障 (T4-open: n = 6/7 开侧否定) -/
2
3  theorem cut_card_bound {α : Type*} [DecidableEq α] {S : List α} {
4    N : ℕ} {lab : α → Fin N}

```

```

4 {G : SimpleGraph α}
5 (hadj : ∀ a b : α, G.Adj a b → a ∉ S → b ∉ S → lab a = lab
6   b) :
7   ∀ {L : List α}, L.IsChain G.Adj →
8     ((L.filter (· ∉ S)).map lab).toFinset.card ≤ 1 + L.tail.
9   countP (· ∈ S) := by
10     intro L
11     induction L with
12     | nil => intro _; simp
13     | cons x xs ih =>
14       intro hL
15       cases xs with
16       | nil => by_cases hx : x ∈ S <;> simp [hx]
17       | cons y ys =>
18         obtain ⟨hxy, htail⟩ := List.isChain_cons_cons.mp hL
19         have hih := ih htail
20         rw [List.tail_cons] at hih
21         rw [List.tail_cons]
22         by_cases hx : x ∈ S <;> by_cases hy : y ∈ S
23         · -- ① x ∈ S, y ∈ S
24           have hF : (x :: y :: ys).filter (· ∉ S) = (y :: ys).
25             filter (· ∉ S) :=
26               List.filter_cons_of_neg (by simp [hx])
27             rw [hF]
28             have hc : (y :: ys).countP (· ∈ S) = ys.countP (· ∈ S)
29             + 1 := by simp [hy]
30             rw [hc]; omega
31         · -- ② x ∈ S, y ∉ S
32           have hF : (x :: y :: ys).filter (· ∉ S) = (y :: ys).
33             filter (· ∉ S) :=
34               List.filter_cons_of_neg (by simp [hx])
35             rw [hF]
36             have hc : (y :: ys).countP (· ∈ S) = ys.countP (· ∈ S)
37             := by simp [hy]
38             rw [hc]; omega
39         · -- ③ x ∉ S, y ∈ S
40           have hF : (x :: y :: ys).filter (· ∉ S) = x :: (y ::
41             ys).filter (· ∉ S) :=
42             List.filter_cons_of_pos (by simp [hx])
43             rw [hF, List.map_cons, List.toFinset_cons]
44             have hc : (y :: ys).countP (· ∈ S) = ys.countP (· ∈ S)
45             + 1 := by simp [hy]
46             rw [hc]
47             exact le_trans (Finset.card_insert_le (lab x) _) (by
48               omega)

```

```

40      · -- ④  $x \notin S, y \notin S$ 
41      have hF : (x :: y :: ys).filter (·  $\notin$  S) = x :: (y ::
ys).filter (·  $\notin$  S) :=
42          List.filter_cons_of_pos (by simp [hx])
43      rw [hF, List.map_cons, List.toFinset_cons]
44      have hc : (y :: ys).countP (·  $\in$  S) = ys.countP (·  $\in$  S)
:= by simp [hy]
45      rw [hc]
46      have hlab : lab x = lab y := hadj x y hxy hx hy
47      have hymem : lab y  $\in$  ((y :: ys).filter (·  $\notin$  S)).map
lab).toFinset :=
48          List.mem_toFinset.mpr (List.mem_map_of_mem (f := lab
)
49              (List.mem_filter.mpr (by simp, by simp [hy])))
50      rw [hlab, Finset.insert_eq_of_mem hymem]
51      omega
52
53 theorem cut_bound {V : Type*} [Fintype V] [DecidableEq V] {G :
SimpleGraph V} {N : ℕ}
54   (S : List V) (hSnd : S.Nodup) (lab : V → Fin N)
55   (hadj : ∀ a b : V, G.Adj a b → a  $\notin$  S → b  $\notin$  S → lab a = lab
b)
56   (hsurj : ∀ i : Fin N, ∃ v : V, v  $\notin$  S ∧ lab v = i)
57   {a b : V} {w : G.Walk a b} (hw : w.IsHamiltonian) :
58   N ≤ S.length + 1 := by
59   have hchain : w.support.IsChain G.Adj := SimpleGraph.Walk.
isChain_adj_support w
60   have hnd : w.support.Nodup :=
61       List.nodup_iff_count_le_one.mpr (fun v => by have h := hw v;
omega)
62   have hcut := cut_card_bound hadj hchain
63   have hcover : ∀ i : Fin N, i  $\in$  ((w.support.filter (·  $\notin$  S)).map
lab).toFinset := by
64       intro i
65       obtain ⟨v, hvS, hvl⟩ := hsurj i
66       have hvm : v  $\in$  w.support := List.count_pos_iff.mp (by have h
:= hw v; omega)
67       rw [← hvl]
68       exact List.mem_toFinset.mpr (List.mem_map_of_mem (f := lab)
(List.mem_filter.mpr (hvm, by simp [hvS])))
69   have hNcard : N = (Finset.univ : Finset (Fin N)).card := by
70       rw [Finset.card_univ, Fintype.card_fin]
71   have hcle : (Finset.univ : Finset (Fin N)).card
72       ≤ ((w.support.filter (·  $\notin$  S)).map lab).toFinset.card :=
73       Finset.card_le_card (fun i _ => hcover i)
74

```

```

75   have htail : w.support.tail.countP (· ∈ S) ≤ w.support.countP (·
      ∈ S) :=
76     List.Sublist.countP_le (List.tail_sublist w.support)
77   have hlen : w.support.countP (· ∈ S) = (w.support.filter (· ∈ S)
      ).length :=
78     List.countP_eq_length_filter
79   have hsub : List.Subperm (w.support.filter (· ∈ S)) S :=
80     List.Nodup.subperm (hnd.filter _)
81     (fun x hx => of_decide_eq_true (List.mem_filter.mp hx).2)
82   have hflen := hsub.length_le
83   omega
84
85   theorem no_hamiltonian_of_cut {V : Type*} [Fintype V] [DecidableEq
      V] {G : SimpleGraph V} {N : ℕ}
86     (S : List V) (hSnd : S.Nodup) (lab : V → Fin N)
87     (hadj : ∀ a b : V, G.Adj a b → a ∉ S → b ∉ S → lab a = lab
      b)
88     (hsurj : ∀ i : Fin N, ∃ v : V, v ∉ S ∧ lab v = i)
89     (hN : S.length + 1 < N) :
90     ¬ ∃ a b : V, ∃ w : G.Walk a b, w.IsHamiltonian := by
91     rintro ⟨a, b, w, hw⟩
92     have h := cut_bound S hSnd lab hadj hsurj hw
93     omega
94
95   /-- n=6 顶点构造助手。 -/
96   def v6 (r : Fin 3) (c : Fin 6) (h : IsCell 6 (r, c) := by decide)
      : V 6 := ⟨(r, c), h⟩
97
98   /-- n=6 割集证书: 中间两列 6 格。 -/
99   def cutS6 : List (V 6) := [v6 0 2, v6 0 3, v6 1 2, v6 1 3, v6 2 2,
      v6 2 3]
100
101   /-- n=6 连通块标签 (S 格与未列格 ↦ 0)。 -/
102   def cutLab6 : V 6 → Fin 8 := fun v =>
103     match v.1.1.val, v.1.2.val with
104     | 0, 1 => 0 | 0, 4 => 1 | 0, 5 => 2
105     | 1, 0 => 3 | 1, 1 => 4 | 1, 4 => 5 | 1, 5 => 6
106     | 2, 0 => 0 | 2, 1 => 7 | 2, 4 => 2
107     | _, _ => 0
108
109   theorem no_open_tour_6 :
110     ¬ ∃ a b : V 6, ∃ w : (knightGraph 6).Walk a b, w.IsHamiltonian
      :=
111     no_hamiltonian_of_cut cutS6 (by decide) cutLab6 (by decide) (by
      decide) (by decide)

```

```

112
113 /-- n=7 顶点构造助手。 -/
114 def v7 (r : Fin 3) (c : Fin 7) (h : IsCell 7 (r, c) := by decide)
115   : V 7 := ⟨(r, c), h⟩
116
117 /-- n=7 割集证书: 5 格。 -/
118 def cutS7 : List (V 7) := [v7 0 2, v7 0 4, v7 1 3, v7 2 2, v7 2 4]
119
120 /-- n=7 连通块标签 (S 格与未列格  $\mapsto 0$ )。 -/
121 def cutLab7 : V 7 → Fin 7 := fun v =>
122   match v.1.1.val, v.1.2.val with
123   | 0, 1 => 0 | 0, 3 => 1 | 0, 5 => 2 | 0, 6 => 3
124   | 1, 0 => 4 | 1, 1 => 1 | 1, 2 => 0 | 1, 4 => 3 | 1, 5 => 1 | 1,
125     6 => 5
126   | 2, 0 => 0 | 2, 1 => 6 | 2, 3 => 1 | 2, 5 => 3
127   | _, _ => 0
128
129 theorem no_open_tour_7 :
130   ¬ ∃ a b : V 7, ∃ w : (knightGraph 7).Walk a b, w.IsHamiltonian
131   :=
132   no_hamiltonian_of_cut cutS7 (by decide) cutLab7 (by decide) (by
133     decide) (by decide)

```

Listing 6: Verbatim lines 2577–2633: the assembly of both iff theorems from the stage lemmas, followed by the two closing `#print axioms` commands.

```

1 /-! ## 16. T5 组装: 双 iff 主定理 (statement 冻结件) -/
2
3 /-- **定理 0** (开巡游 iff) :  $K_n$  存在哈密顿路径  $\iff n = 1 \vee 4 \vee 5 \vee \geq 8$ 。 -/
4 theorem exists_hamiltonian_path_iff (n : ℕ) :
5   (∃ a b : V n, ∃ w : (knightGraph n).Walk a b, w.IsHamiltonian)
6   ↔
7   n = 1 ∨ n = 4 ∨ n = 5 ∨ 8 ≤ n := by
8   constructor
9   · intro h
10    by_cases h8 : 8 ≤ n
11    · exact Or.inr (Or.inr (Or.inr h8))
12    · push_neg at h8
13      have : n = 0 ∨ n = 1 ∨ n = 2 ∨ n = 3 ∨ n = 4 ∨ n = 5 ∨ n =
14        6 ∨ n = 7 := by omega
15      rcases this with rfl | rfl | rfl | rfl | rfl | rfl | rfl |
16      rfl
17      · exact absurd h no_open_tour_0
18      · exact Or.inl rfl

```

```

16 · exact absurd h no_open_tour_2
17 · exact absurd h no_open_tour_3
18 · exact Or.inr (Or.inl rfl)
19 · exact Or.inr (Or.inr (Or.inl rfl))
20 · exact absurd h no_open_tour_6
21 · exact absurd h no_open_tour_7
22 · rintro (rfl | rfl | rfl | h8)
23 · exact open_tour_1
24 · exact open_tour_4
25 · exact open_tour_5
26 · rcases Nat.even_or_odd n with he | ho
27 · exact open_of_closed (closed_cycle_exists n he h8)
28 · obtain ⟨k, rfl⟩ := ho
29 · exact open_tour_exists_odd _ (by omega) ⟨k, rfl⟩
30
31 /-- **定理 C** (闭巡游 iff) :  $K_n$  存在哈密顿圈  $\iff$   $\text{Even } n \wedge \geq 8$ .
    -/
32 theorem exists_hamiltonian_cycle_iff (n : ℕ) :
33   (∃ a : V n, ∃ w : (knightGraph n).Walk a a, w.
34     IsHamiltonianCycle)  $\leftrightarrow$ 
35     Even n  $\wedge$  8 ≤ n := by
36   constructor
37   · intro h
38     constructor
39     · rcases Nat.even_or_odd n with he | ho
40       · exact he
41       · exact absurd h (no_cycle_odd ho)
42     · by_contra h8
43       push_neg at h8
44       have : n = 0  $\vee$  n = 1  $\vee$  n = 2  $\vee$  n = 3  $\vee$  n = 4  $\vee$  n = 5  $\vee$  n =
45         6  $\vee$  n = 7 := by omega
46       rcases this with rfl | rfl | rfl | rfl | rfl | rfl | rfl |
47       rfl
48       · exact no_closed_tour_0 h
49       · exact no_cycle_odd ⟨0, rfl⟩ h
50       · exact no_closed_tour_2 h
51       · exact no_cycle_odd ⟨1, rfl⟩ h
52       · exact no_closed_tour_4 h
53       · exact no_cycle_odd ⟨2, rfl⟩ h
54       · exact no_closed_tour_6 h
55       · exact no_cycle_odd ⟨3, rfl⟩ h
56 · rintro ⟨he, h8⟩
57   exact closed_cycle_exists n he h8
58
59 #print axioms exists_hamiltonian_path_iff

```

```
57 #print axioms exists_hamiltonian_cycle_iff
```

4 Verification evidence

- Toolchain: Lean v4.34.0 (leanprover/lean4:v4.34.0), mathlib4 v4.34.0, revision 5ed29652. Reproduction: with that pin, `lake env lean` on `proofs/01-holestrip-tour-proved.lean` compiles with exit code 0.
- Statement integrity: the frozen statement file (84 lines) has sha256

```
96875db4a784ff7a3862e3fe255c9e25c09f98
a68d9d98d30700d90b4fd6929e,
```

byte-identical to the frozen Lean file `holestrip-tour.lean`; the final proof file (130,382 bytes) has sha256

```
9cf0b5b0af8cedb2edfd9d63779a2e8d4a6c973
237b616c1482a05bbf1e0a70b.
```

A re-run of the audit department's symmetric-difference method (comments stripped, declarations split, definitions compared full-block, theorems up to `:=`) reports all eight frozen declarations header-identical (six definitions and the two main theorems), with no frozen-only declaration and proof-body scaffolding added on the final side (`audit/statement-diff.out`).

- Kernel check: compiles with 0 sorry (whole-file scan including comments, also covering `admit` and `native_decide`: zero hits); `#print axioms` output, verbatim (kernel re-run of 2026-09-30, exit code 0, kept in `audit/axioms.out`):

```
1 'open_tour_exists_odd' depends on axioms: [propept, Classical.
    choice, Quot.sound]
2 'open_good_list' depends on axioms: [propept, Classical.choice,
    Quot.sound]
3 'step_open_list' depends on axioms: [propept, Classical.choice,
    Quot.sound]
4 'closed_cycle_exists' depends on axioms: [propept, Classical.choice
    , Quot.sound]
5 'openBase9V_good' depends on axioms: [propept, Classical.choice,
    Quot.sound]
```

```

6 'openBase11V_good' depends on axioms: [propext, Classical.choice,
    Quot.sound]
7 'card_V' depends on axioms: [propext, Classical.choice, Quot.sound]
8 'no_cycle_odd' depends on axioms: [propext, Classical.choice, Quot.
    sound]
9 'no_open_tour_2' depends on axioms: [propext, Classical.choice,
    Quot.sound]
10 'no_closed_tour_2' depends on axioms: [propext, Classical.choice,
    Quot.sound]
11 'no_open_tour_3' depends on axioms: [propext, Classical.choice,
    Quot.sound]
12 'no_closed_tour_3' depends on axioms: [propext, Classical.choice,
    Quot.sound]
13 'open_tour_1' depends on axioms: [propext, Classical.choice, Quot.
    sound]
14 'no_open_tour_0' depends on axioms: [propext, Quot.sound]
15 'no_closed_tour_0' depends on axioms: [propext, Quot.sound]
16 'no_closed_tour_4' depends on axioms: [propext, Classical.choice,
    Quot.sound]
17 'no_closed_tour_6' depends on axioms: [propext, Classical.choice,
    Quot.sound]
18 'exists_hamiltonian_path_iff' depends on axioms: [propext,
    Classical.choice, Quot.sound]
19 'exists_hamiltonian_cycle_iff' depends on axioms: [propext,
    Classical.choice, Quot.sound]

```

The two main theorems depend on exactly {propext, Classical.choice, Quot.sound}; every line is a subset of the whitelist.

- Grading by the audit department: both proven theorems *complete proof*; the lowest grade reached for the case = *complete proof* (final adjudication `gate3-final-audit.md`: statement symmetric difference 8/8 identical, strict compile exit 0, zero `sorry/admit`, axiom whitelist subset, no added hypotheses). The adjudication also records the non-blocking disclosures carried into Section 6.
- Audit chain (originals in this deposit under `audit/` unless noted): the final adjudication, the symmetric-difference re-run, the axiom re-print, and this note's claim check `claim-check.md` — added to `audit/` after the audit department returns its adjudication, and a prerequisite for publication.

5 Novelty evidence

Adjudication copied from the pipeline’s exclusivity reviews (C9 gate), four rounds of which ran before the problem was accepted and during it: verdict **no occupying record found** for the existence characterisation of open and closed tours on K_n . Value tier: a small original classification theorem. The nearest published neighbours are delimited explicitly and none of them is this family: Miller–Farnsworth [3] classify closed tours on $3 \times n$ boards with *one* square removed; Srichote et al. [4] classify closed tours on $4 \times n$ boards with *two* squares removed; Bi et al. [5] raised the two-square question for $4 \times n$ only; DeMaio–Hippchen [6] give the minimal number of squares to remove, not positions; Schwenk [7] and the classical line treat full boards. The registered counting sequence for full $3 \times n$ boards, OEIS A169696 [9], is a different object (no holes; counting, not existence). The full query log, including every zero-hit query, is in this deposit at `audit/c9-record.md` (four source records, byte-for-byte).

source	audit	audit
Miller–Farnsworth [3] classify closed tours on $3 \times n$ boards with <i>one</i> square removed; Srichote et al. [4] classify closed tours on $4 \times n$ boards with <i>two</i> squares removed; Bi et al. [5] raised the two-square question for $4 \times n$ only; DeMaio–Hippchen [6] give the minimal number of squares to remove, not positions; Schwenk [7] and the classical line treat full boards. The registered counting sequence for full $3 \times n$ boards, OEIS A169696 [9], is a different object (no holes; counting, not existence). The full query log, including every zero-hit query, is in this deposit at <code>audit/c9-record.md</code> (four source records, byte-for-byte).	no occupying record found for the existence characterisation of open and closed tours on K_n . Value tier: a small original classification theorem. The nearest published neighbours are delimited explicitly and none of them is this family: Miller–Farnsworth [3] classify closed tours on $3 \times n$ boards with <i>one</i> square removed; Srichote et al. [4] classify closed tours on $4 \times n$ boards with <i>two</i> squares removed; Bi et al. [5] raised the two-square question for $4 \times n$ only; DeMaio–Hippchen [6] give the minimal number of squares to remove, not positions; Schwenk [7] and the classical line treat full boards. The registered counting sequence for full $3 \times n$ boards, OEIS A169696 [9], is a different object (no holes; counting, not existence). The full query log, including every zero-hit query, is in this deposit at <code>audit/c9-record.md</code> (four source records, byte-for-byte).	no occupying record found for the existence characterisation of open and closed tours on K_n . Value tier: a small original classification theorem. The nearest published neighbours are delimited explicitly and none of them is this family: Miller–Farnsworth [3] classify closed tours on $3 \times n$ boards with <i>one</i> square removed; Srichote et al. [4] classify closed tours on $4 \times n$ boards with <i>two</i> squares removed; Bi et al. [5] raised the two-square question for $4 \times n$ only; DeMaio–Hippchen [6] give the minimal number of squares to remove, not positions; Schwenk [7] and the classical line treat full boards. The registered counting sequence for full $3 \times n$ boards, OEIS A169696 [9], is a different object (no holes; counting, not existence). The full query log, including every zero-hit query, is in this deposit at <code>audit/c9-record.md</code> (four source records, byte-for-byte).

The channels are named so a later reader can re-run them; if any channel later returns a same-form record, the tier above weakens accordingly, and this deposit will be amended by a later version rather than by editing this record. Residual risks are ledgered in the C9 record (the Jelliss 2019 monograph and Chessics issues were checked at index level only; the Van Rees classroom notes were reachable at bibliographic level only; the Stack-Exchange API was rate-limited during one round; Thai-language literature was checked through the journal’s own site).

6 Scope and limitations

(a) The kernel results are Theorems 1 and 2 and nothing else. (b) The counting companion of the same family (the number of open/closed tours of K_n) is auxiliary computational data of the source task; it is not claimed here and not submitted to OEIS (the historical OEIS sequence queries of the novelty record, all zero-hit, are included in the record verbatim), and it is not part of this note. (c) The case $n = 1$ of Theorem 1 comes from the pre-freeze correction F0b (recorded in the task card). (d) Scaffolding disclosure, from the final adjudication: the cut-barrier section was drafted by a pipeline worker run and integrated by the main agent (four integration-level repairs); the final assembly section was written by the main agent after a scratch

pre-run. This affects provenance, not the grading: the graded object is the final kernel-checked artifact. (e) The delivered file relaxes elaboration resources (`maxHeartbeats` 4,000,000 and `maxRecDepth` 8,000) before the cut-barrier section; this affects elaboration time limits only, not the kernel trust base. (f) An earlier search-based route to the $n = 6, 7$ impossibility (a decidability checker) was abandoned after a resource-exhaustion probe and is kept in the source task’s archive; the delivered route is the cut certificate. (g) The novelty search is scoped to reachable channels; the residual risks listed in Section 5 are carried rather than resolved. (h) All listed listings are byte-exact contiguous excerpts of the frozen artifacts (LF-normalised where the source has CRLF-dominant mixed endings; the byte-exact originals are shipped in `proofs/`).

Data availability

The Lean sources (`proofs/`: frozen statement file, final proof file, `lean-toolchain`), the verification and audit logs, and the full novelty-search record are packed in the Zenodo deposit of this note (DOI reserved at upload). The note is under CC BY 4.0, the code under MIT. The upload steps are listed in `UPLOAD.md` in the source repository (not shipped in this deposit).

Use of generative AI

(a) AI performed: candidate generation and the four-round exclusivity review; autoformalisation of the definitions and statements; the roundtrip semantic check; proof search and proof-script writing (worker agents, with a main-agent integration round on the cut-barrier section as recorded in the ledger); error classification and repair; the computational exploration scripts; and the assembly of this note including its verbatim listings. (b) Model, node and budget: the proving phase ran on the node designated by the author for each session (`anthropic/stepfun/step-5-preview` after a mid-task switch directed by the author; `anthropic/a6api-main/kimi-k3` before it), consuming 29 agent-run samples of a permitted 36; 4 llm-calls of a permitted 6 (one node smoke plus three roundtrip attempts that failed at the transport layer, after which the semantic roundtrip was re-routed through the agent channel and ledgered under the sampling column); 49 strict kernel compiles of a permitted 56, plus compile-debug probes ledgered separately; this note’s lane consumed 0 pipeline LLM calls against a cap of

4 and three formal LaTeX rounds of a permitted ten (two assembly rounds plus one audit-fix round), with three compile-debug render probes and one kernel re-verification of the shipped axioms listing ledgered separately; the assembly node was `anthropic/a6api-main/deepseek-v4.1-flash`. (c) Final adjudication: all formal statements and proofs reported in this note were checked by the Lean 4 kernel (Lean v4.34.0, mathlib v4.34.0): every proof compiles with no `sorry`, and `#print axioms` reports only `{propext, Quot.sound, Classical.choice}`. (d) The AI system is not an author; the human author reviewed the evidence and is responsible for all content.

Author contributions

CRediT-style: the human author adjudicated topic selection, statement semantics, sign-off and deposit decisions; the AI4Math pipeline (an LLM) generated the candidate proposition, the proof searches, the computational exploration scripts and the text drafts. The AI system is not listed as an author.

References

- [1] L. de Moura and S. Ullrich. The Lean 4 theorem prover and programming language. *CADe-28*, 2021. https://doi.org/10.1007/978-3-030-79876-5_37
- [2] The mathlib Community. The Lean mathematical library. *CPP 2020*, 2020. <https://doi.org/10.1145/3372885.3373824>
- [3] J. Miller and D. Farnsworth. Knight’s tours on $3 \times n$ chessboards with a single square removed. *Open J. Discrete Math.* 3(1):56–59, 2013. <https://doi.org/10.4236/ojdm.2013.31012>
- [4] W. Srichote, R. Boonklurb, T. Kaewwannarat, and S. Singhun. Closed knight’s tours on $4 \times n$ chessboards with two squares removed. *Thai J. Math.*, Special Issue: Annual Meeting in Mathematics 2021, 64–81, 2022. (Zbl 1538.05160)
- [5] Z. Bi, S. Butler, A. DeGraaf, and E. Doebel. Knight’s tours on boards with odd dimensions. *Involve* 8(4):615–627, 2015. <https://doi.org/10.2140/involve.2015.8.615>

- [6] J. DeMaio and T. Hippchen. Closed knight's tours with minimal square removal for all rectangular boards. *Math. Mag.* 82(3):219–225, 2009. <https://doi.org/10.1080/0025570X.2009.11953624>
- [7] A. J. Schwenk. Which rectangular chessboards have a knight's tour? *Math. Mag.* 64(5):325–332, 1991. <https://doi.org/10.1080/0025570X.1991.11977627>
- [8] OEIS Foundation Inc. The On-Line Encyclopedia of Integer Sequences. <https://oeis.org>
- [9] OEIS Foundation Inc. Sequence A169696: Number of undirected knight's tours on a $3 \times n$ board. <https://oeis.org/A169696>